

Implementation methodology and bring-up strategy

How the 4×4 weight-stationary systolic-array multiplier is taken from RTL to running silicon on a ZedBoard — with the VIO/ILA method as the primary path, the exact protocol used to inject data, the exact way results are read back, and every report checked before sign-off

Target part `xc7z020clg484-1` · Vivado 2019.1 or later (measurements from 2020.1) · Design under test

`accelerator_system_top` — **never modified**

Harness `rtl/vio_debug_top.sv` · Constraints `vio_debug.xdc` · Automation `vio_demo.tcl` · Companion document: `timing.pdf`

Contents

1. Methodology in one page
2. The governing constraint: the RTL is frozen
3. The four candidate methods and why VIO + ILA was chosen
4. The two impedance mismatches the harness must solve
5. Harness architecture — `vio_debug_top.sv`
6. The VIO core — exact configuration
7. The ILA core — exact configuration
8. Non-invasive internal probing via XDC
9. The build flow, step by step
10. Strategy for inputting data
11. Data packing — the lane and address rules
12. Strategy for watching results
13. Scripted operation — `vio_demo.tcl` dissected
14. The expected ILA cycle map
15. Trigger recipes — what to look at and when
16. Parameters and reports checked at each stage
17. The pass/fail criteria we actually apply
18. Failure modes and what each report says
19. Alternative methods 1, 3 and 4 — when to use them
20. Sign-off checklist

1. Methodology in one page

The project is verified in simulation first and only then moved to hardware, and the hardware run is deliberately made **comparable** to the simulation run rather than being a new, separate experiment. That is the whole strategy, and everything else follows from it:

1. RTL FROZEN	13 SystemVerilog files. Never edited by any method.
2. SIMULATION BASELINE	QuestaSim regression: 11/11 testbenches, 3 680 checks, 0 mismatches. T5 golden case established as the vector that will be replayed on hardware.
3. HARNESS, NOT SURGERY	A wrapper (<code>vio_debug_top</code>) is added AROUND the design. It drives the same <code>host_*</code> ports the testbench drove.
4. BUILD + STATIC CHECKS	<code>synth</code> → utilization → <code>impl</code> → TIMING (WNS/WHS/WPWS) → DRC → methodology → bitstream. See <code>timing.pdf</code> .
5. INJECT OVER JTAG	VIO <code>probe_out</code> drives weights, activations, control. No board switches, no serial port, no external pins.
6. OBSERVE TWO WAYS	VIO <code>probe_in</code> = the ANSWER (did it compute correctly?) ILA waveform = the PROCESS (did it compute correctly for the right reason, cycle by cycle?)
7. COMPARE TO GOLDEN	Same T5 matrices as simulation → same expected numbers. Automated by <code>vio_demo.tcl</code> , which prints PASS / MISMATCH.

Step 6 is the part that makes this methodology worth writing down. Reading a correct answer out of a VIO proves the design produced the right numbers. It does **not** prove the design produced them the way the specification says it should — a design could produce `28 32 36 40` by accident, or by a path that will break on the next input. The ILA

closes that gap by capturing the actual 17-cycle execution and letting you check it against a cycle map derived from the RTL: weight load at cycles 1–4, one activation per clock in compute, results landing at a fixed 8-cycle store latency. **Answer plus process. Neither alone is sufficient.**

2. The governing constraint: the RTL is frozen

The 13 design files in `...SIMPLE_ACCEL_RTL/` are the verified artefact. They carry a regression result — 11/11 testbenches, 3 680 checks, 0 mismatches — and that result is only meaningful if the thing running on the board is bit-identical to the thing that was simulated. So:

Rule	Consequence for the methodology
No edits to the 13 RTL files	Every method <i>adds a wrapper</i> . <code>vio_debug_top</code> , <code>fpga_top</code> and <code>accel_axi</code> all instantiate <code>accelerator_system_top</code> unmodified and drive its existing <code>host_*</code> ports.
No (* mark_debug *) attributes in the source	Internal nets are marked for the ILA from the XDC instead — <code>set_property MARK_DEBUG true [get_nets ...]</code> . Same effect, zero source changes. See §8.
No debug-only ports added to the design	Anything the harness needs that the accelerator does not expose is derived <i>outside</i> it: <code>done_sticky</code> and <code>done_count</code> are built in the wrapper from the existing <code>done</code> pulse.
The host protocol is honoured, not worked around	Fixed latency, no back-pressure, weight rows in natural order, <code>start</code> only when <code>busy</code> is low. The harness respects the same contract the testbench did.

WHY THIS MATTERS MORE THAN IT LOOKS

If the hardware build differed from the simulated build, then a hardware mismatch would have two possible causes — a real hardware problem, or the edit — and you could not tell them apart. Freezing the RTL means

ANY MISMATCH ON THE BOARD IS UNAMBIGUOUSLY A HARDWARE OR SETUP PROBLEM

. That is what makes the bring-up debuggable.

3. The four candidate methods and why VIO + ILA was chosen

#	Method	Top module	You interact via	Verdict
1	Switches + LEDs	<code>fpga_top</code>	On-board SW / BTN / LED	Fully self-contained demo — no PC needed once programmed. But 8 switches and 8 LEDs cannot hold a 4×4 int8 matrix, so it needs a console FSM and 32 button presses to enter one job. Tedious, error-prone, and shows you <i>nothing</i> about the pipeline.
2	VIO + ILA over JTAG	<code>vio_debug_top</code>	Vivado Hardware Manager	Chosen. One physical pin. Data injection and result readback from the PC. Scriptable in Tcl. The only method that can watch the pipeline cycle-by-cycle on real silicon.
3	USB-UART console	BD wrapper (<code>accel_axi</code> + PS7)	Typed text in a terminal	Excellent ergonomics, but on the ZedBoard the USB-UART is a PS resource (UART1, MIO48/49), so it drags in a Zynq PS, an AXI interconnect, Vitis, a BSP and a C application. Large tooling cost for a PL-only accelerator, and still no cycle-level visibility.
4	Full PS/AXI integration	BD wrapper + DMA/IRQ	Driver or Linux app	The right answer for a <i>product</i> — DMA streaming, done-interrupt, UIO driver. Overkill for bring-up, and it hides the pipeline behind a software stack.

The decision, stated as criteria

Criterion	M1	M2 (VIO/ILA)	M3	Why it mattered
Physical pins required	~22	1 (clk)	0 (PS pins)	Fewer pins = fewer XDC mistakes = fewer board-revision hazards.
Can enter a full 4×4 × 4×4 job quickly	32 button presses	8 Tcl calls	Typed text	Human error during data entry is the dominant bring-up failure mode.
Repeatable / scriptable	No	Yes — Tcl	Partly	A regression you can re-run in one command is a regression you will actually re-run.
Cycle-accurate hardware visibility	None	Yes — ILA	None	The whole point. A 170 ns job is invisible to any human-rate interface.
Extra tooling needed	None	Vivado only	Vivado + Vitis	Vitis, a platform project and a BSP are a lot of moving parts to debug alongside the accelerator.
Cost	Free	~0.565 ns of WNS, ~8 BRAM36	PS + interconnect	Observability is not free — see timing.pdf §15.

4. The two impedance mismatches the harness must solve

A VIO dashboard is driven by a human clicking a mouse. The accelerator is a 100 MHz pipeline that finishes a job in **17 clocks = 170 ns**. Those two worlds are about six orders of magnitude apart in time, and connecting them naively fails in two specific ways. Understanding these two problems is understanding the harness.

Mismatch 1 — VIO drives LEVELS; the accelerator wants PULSES

A VIO `probe_out` holds whatever value you set until you set it again — typically milliseconds later. But `start`, `host_w_we` and `host_a_we` are **single-cycle strobes**. If `start` were simply tied to a VIO output held high:

```
1 ms held high ÷ 10 ns per clock = 100 000 clocks of continuous start  
→ the FSM relaunches the 17-cycle job roughly 4 000 times per click  
→ done_count jumps by thousands, the ILA retriggers forever, nothing is observable
```

Solution: a `oneshot` edge detector converts every 0→1 transition on a VIO level into exactly one clock of pulse:

```
module oneshot (input logic clk, input logic rst_n, input logic lvl, output logic pulse);  
    logic lvl_q;  
    always_ff @(posedge clk) begin  
        if (!rst_n) begin lvl_q <= 1'b0; pulse <= 1'b0; end  
        else          begin lvl_q <= lvl;  pulse <= lvl & ~lvl_q; end  
    end  
endmodule
```

Three instances exist — `u_os_start`, `u_os_wwe`, `u_os_awe`. Note the operating consequence: **every write is a 0→1→0 sequence, not a single set**. You must drive the level back to 0 to re-arm the edge detector for the next write. This is why every write helper in the Tcl script ends with a `... 0` call.

Mismatch 2 — `done` is a 10 ns pulse; VIO samples at human rate

`done` is asserted for exactly one clock in the `PH_DONE` state. A VIO reads the fabric only when you click *Refresh* or when Vivado polls — hundreds of milliseconds apart. The probability of a refresh landing on that single 10 ns window is essentially zero. **You will never see `done` on a VIO.**

Solution: latch it in the wrapper, and add a counter:

```
always_ff @(posedge clk) begin  
    if (!rst_n) begin done_sticky <= 1'b0; done_count <= '0; end  
    else begin  
        if (start_p) done_sticky <= 1'b0; // arm for the new job  
        else if (done) done_sticky <= 1'b1; // latch the 10 ns pulse  
        if (done) done_count <= done_count + 1'b1;  
    end  
end
```

Signal	What it answers	Why it is designed this way
<code>done_sticky</code>	"Has the job I just launched finished?"	Set by <code>done</code> , cleared by the next <code>start_p</code> — so it is self-arming. You never have to remember to reset it manually, which removes an entire class of "stale sticky bit" confusion.
<code>done_count</code>	"How many jobs have ever completed?"	The cheapest possible health probe. If it increments by exactly 1 per start click, the one-shot works and the FSM is healthy. If it jumps by thousands, the one-shot was optimised away or bypassed. If it does not move at all, the job never launched.

THE GENERAL LESSON

Whenever you bridge a slow observer to a fast machine, you need two pieces of glue in opposite directions:

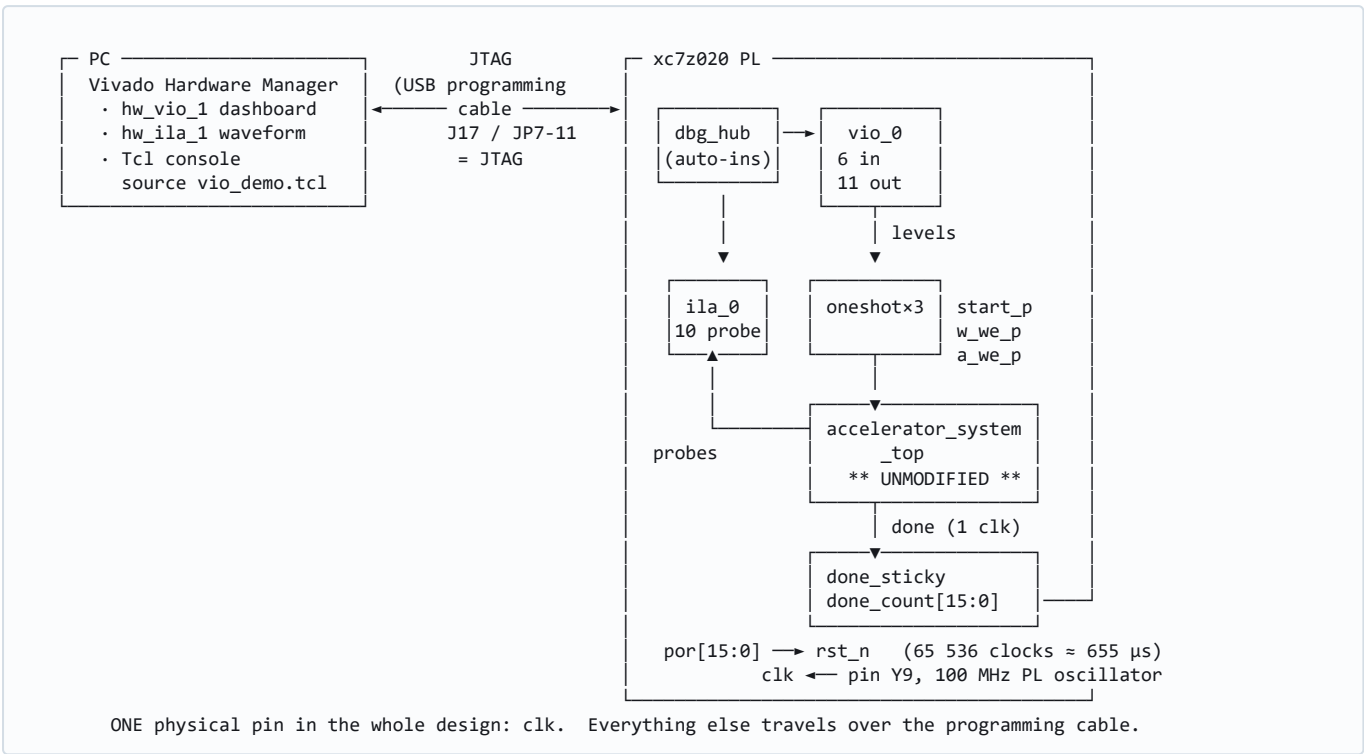
EDGE-DETECT THE SLOW SIDE'S LEVELS

so the fast side sees events, and

LATCH THE FAST SIDE'S EVENTS

so the slow side can see them. Every VIO harness you ever build will need both.

5. Harness architecture — vio_debug_top.sv



Harness element	Purpose
Power-on reset counter	A 16-bit counter holds <code>rst_n</code> low for 65 536 clocks ($\approx 655 \mu\text{s}$) after configuration, then releases synchronously. The declaration initialiser is a synthesisable power-up value on 7-series — flip-flops configure to a known state at bitstream load, so no external reset pin is needed. 655 μs is long past by the time your first JTAG transaction lands.
3 × oneshot	Level → single-clock pulse. See §4.
accelerator_system_top	The frozen DUT. Ports driven exactly as the testbench drove them.
done_sticky / done_count	Fast event → observable state. See §4.
result_flat	<code>result</code> is an unpacked SystemVerilog array <code>[0:3]</code> of int32. VIO and ILA probes take packed vectors, so it is flattened: <code>{result[3], result[2], result[1], result[0]}</code> → 128 bits.
vio_0	Injection and readback. §6.
ila_0	Cycle-by-cycle capture. §7.

6. The VIO core — exact configuration

Generate from the IP catalog as `vio_0`, VIO (Virtual Input/Output) v3.0, with **6 input probes and 11 output probes**, and *Enable Input Probe Activity Detectors* ticked.

6.1 probe_in — what the dashboard shows you

Port	Signal	Width	What you learn from it
probe_in0	busy	1	Read this before every start. Must be 0 or the start pulse is ignored.
probe_in1	done_sticky	1	Job complete. Auto-cleared by the next start.
probe_in2	phase	3	FSM state: 0 IDLE / 1 WLOAD / 2 COMPUTE / 3 DRAIN / 4 DONE. At human rate you will essentially always see 0.
probe_in3	o_rdata	128	The result readback path. Four int32 lanes = one output row.
probe_in4	result_flat	128	The array's live result bus — the last row that came out of the de-skew buffer, before it was stored.
probe_in5	done_count	16	Health probe. +1 per start click, exactly.

6.2 probe_out — what you drive

Port	Signal	Width	Role
probe_out0	vio_start_lvl	1	→ oneshot → start . Launch a job.
probe_out1	vio_num_vectors	8	How many activation vectors this job processes. Set initial value 0x04 .
probe_out2	vio_act_base	8	Base address in a_mem to stream from.
probe_out3	vio_out_base	8	Base address in output_mem to write to.
probe_out4	vio_w_wdata	32	One packed weight row (4 × int8).
probe_out5	vio_w_addr	2	Weight row index 0–3. 2 bits because weight_mem is only 4 deep.
probe_out6	vio_w_we_lvl	1	→ oneshot → host_w_we . Commit the weight word.
probe_out7	vio_a_wdata	32	One packed activation vector (4 × int8).
probe_out8	vio_a_addr	8	Activation index 0–255. 8 bits because a_mem is 256 deep.
probe_out9	vio_a_we_lvl	1	→ oneshot → host_a_we . Commit the activation word.
probe_out10	vio_o_addr	8	Which output_mem row to present on o_rdata .

THE ASYMMETRIC ADDRESS WIDTHS ARE DELIBERATE, AND THEY ENCODE THE ARCHITECTURE

vio_w_addr is

2 BITS

and vio_a_addr is

8 BITS

. That is not an oversight — it is the weight-stationary dataflow made visible in the interface. The weight matrix is loaded **ONCE**

and held stationary in the array (4 rows, hence 2 bits), while activations are a

STREAM

of up to 256 vectors reused against that one weight load. Both take 4 writes for this 4×4 demo, but the activation bus is sized for the real workload.

7. The ILA core — exact configuration

Generate as `ila_0` with **10 probes**, native (not AXI) interface, clocked by `clk`.

Probe	Signal	Width	Why it is captured
probe0	<code>start_p</code>	1	The trigger. Condition <code>probe0 == 1</code> catches the whole job.
probe1	<code>busy</code>	1	Job window: high from WLOAD through DRAIN.
probe2	<code>done</code>	1	The real 1-cycle pulse — the ILA <i>can</i> see it, unlike the VIO.
probe3	<code>phase</code>	3	The most valuable trace in the capture. Lets you measure every phase length directly.
probe4	<code>w_we_p</code>	1	Confirms exactly four 1-clock weight-write spikes.
probe5	<code>a_we_p</code>	1	Confirms exactly four 1-clock activation-write spikes.
probe6	<code>vio_a_addr</code>	8	Which activation slot each write landed in.
probe7	<code>o_rdata</code>	128	Result readback bus.
probe8	<code>result_flat</code>	128	Watch the wavefront emerge here — results appear column by column.
probe9	<code>done_count</code>	16	Distinguishes "this job" from "a previous job" in the capture.

7.1 Core settings and the depth trade-off

Setting	Value	Reasoning
Sample Data Depth	1024 or 256	1024 gives 10.24 μ s of trace at 100 MHz — generous pre/post-trigger context for a 17-cycle job. But $288 \text{ probe bits} \times 1024 = 294\,912 \text{ bits} \approx \mathbf{8 \text{ BRAM36}}$, which is more than three times the accelerator's own memory footprint. 256 is recommended: still ample for a 17-cycle job, and it saves $\approx 6 \text{ BRAM36}$ and the routing pressure that comes with them.
Number of Comparators	1 per probe	Enough for the single-condition triggers in §15.
Trigger Out / In	Off	No cross-triggering with a processor here.
Input Pipe Stages	0	Extra pipe stages shift probes <i>relative to each other</i> and would corrupt the cycle map in §14 — the one thing you are trying to read.
Capture Control	Off (or basic)	The job is short and dense; there is nothing worth filtering out.
Clock domain	<code>clk</code>	Must match the fabric clock. Confirm in <i>Set Up Debug</i> .

8. Non-invasive internal probing via XDC

To see *inside* the accelerator — the AGU strobes, the array's partial sums — you need `MARK_DEBUG` on internal nets. Since the RTL cannot be edited, this is done from the constraint file:

```
# 1. Synthesize once, open the Synthesized Design, find the REAL net names:
get_nets -hierarchical *agu_wload*
get_nets -hierarchical *agu_valid_in*
get_nets -hierarchical *psum*

# 2. Then mark them from the XDC and re-run synthesis:
set_property MARK_DEBUG true [get_nets u_acc/agu_wload]
set_property MARK_DEBUG true [get_nets u_acc/agu_valid_in]
set_property MARK_DEBUG true [get_nets u_acc/agu_out_we]
set_property MARK_DEBUG true [get_nets {u_acc/agu_w_addr[*]}]
set_property MARK_DEBUG true [get_nets {u_acc/agu_act_addr[*]}]
set_property MARK_DEBUG true [get_nets {u_acc/agu_out_addr[*]}]
```

Net	What it proves when you see it
<code>agu_wload</code>	The weight-load enable — exactly 4 cycles wide.
<code>agu_w_addr</code>	Counts DOWN 3, 2, 1, 0. This is the bottom-first weight rule made visible. The host writes rows in natural order; the AGU reverses them.
<code>agu_valid_in</code>	One cycle per activation vector during COMPUTE.
<code>agu_out_we</code> / <code>agu_out_addr</code>	Four write strobes arriving 8 cycles (STORE_LATENCY) after each activation address was issued — the latency derivation, confirmed on silicon.

PREREQUISITE AND ITS COST

Hierarchical net names only survive if synthesis is run with `-flatten_hierarchy none`. That disables cross-boundary optimisation and genuinely costs QoR. With a measured baseline of WNS +0.714 ns on the debug build there is room for it, but

RE-CHECK TIMING AFTER CHANGING IT RATHER THAN ASSUMING

— see [timing.pdf](#) §15. Use it while debugging; drop it for a final build.

9. The build flow, step by step

#	Step	Detail and what to verify
1	Create project	RTL Project, part <code>xc7z020c1g484-1</code> (or the ZedBoard board file).
2	Add sources	The 13 <code>.sv</code> files from <code>...SIMPLE_ACCEL_RTL/</code> plus <code>FPGA_IMPL/rtl/vio_debug_top.sv</code> .
3	Generate <code>vio_0</code>	IP catalog → VIO. 6 probe_in / 11 probe_out with the widths in §6. Set <code>vio_num_vectors</code> initial value to <code>0x04</code> .
4	Generate <code>ila_0</code>	IP catalog → ILA. 10 probes with the widths in §7. Depth 1024 (or 256).
5	Set top	<code>vio_debug_top</code> → <i>Set as Top</i> . It should go bold with no missing-module errors — if <code>vio_0</code> / <code>ila_0</code> are black boxes, the IP was not generated.
6	Add constraints	<code>FPGA_IMPL/vio_debug.xdc</code> . Verify pin Y9 against the Digilent master XDC for your board revision.
7	Run Synthesis	Then check: utilization, synthesis warnings (target: 0 errors / 0 critical warnings), and that the <code>oneshot</code> instances survived (search the netlist for <code>u_os_start</code>).
8	Run Implementation	Then check: Design Timing Summary — WNS, WHS, WPWS all positive, 0 failing endpoints , plus DRC and Methodology. Full detail in <code>timing.pdf</code> .
9	Generate Bitstream	The XDC's <code>UNCONSTRAINEDPINS {Allow}</code> lets a PL-only Zynq build complete. Note the <code>.ltx</code> probes file path.
10	Program	<i>Open Hardware Manager</i> → <i>Open Target</i> → <i>Auto Connect</i> → right-click the <code>xc7z020_1</code> device (not <code>arm_dap_0</code>) → <i>Program Device</i> . Set the Debug probes file to the <code>.ltx</code> or no dashboards will appear.
11	Verify dashboards	<code>hw_vio_1</code> and <code>hw_ila_1</code> must both appear under the device. If not, see §18.
12	Run the demo	Tcl console: <code>cd <path>/FPGA_IMPL, source vio_demo.tcl, accel_demo</code> .

Board setup — the physical prerequisites

Item	Setting
USB cable	J17 (the PROG/JTAG port) — not J14 (UART).
Boot-mode jumpers	JP7–JP11 all to GND = JTAG mode.
JP2	JTAG mode.
Power	PGOOD LED lit.
Drivers	Vivado cable drivers installed; the device must enumerate under <i>Auto Connect</i> .

10. Strategy for inputting data

All data enters through the `host_* write ports` of `accelerator_system_top` — the same ports the simulation testbench used. There are two memories to fill and one job to launch.

10.1 The write protocol — three steps per word, always in this order

For each word to write:

1. set `vio_?_wdata` = the packed 32-bit word (data first)
2. set `vio_?_addr` = the destination index (then address)
3. set `vio_?_we_lvl` = 1 → oneshot emits ONE clock of `host_?_we`
set `vio_?_we_lvl` = 0 → re-arm the edge detector for the next write

Data and address are LEVELS and are already stable when the strobe fires.
The strobe is an EDGE. Order matters; the 1→0 return is mandatory, not cosmetic.

10.2 The four preconditions

Precondition	Why, and what happens if you ignore it
busy must be 0	Host writes are masked while the accelerator is running, and a <code>start</code> arriving while <code>busy</code> is high is silently ignored . This is the single most common cause of "all results read 0". Read <code>busy</code> before every write and every start.
rst_n must have released	The POR is 65 536 clocks $\approx$ 655 μ s. A JTAG round trip is milliseconds, so in practice this is always satisfied — but it is why "busy reads 1 forever" is never a "wait longer" problem.
Weight rows in NATURAL order 0,1,2,3	The design loads weights bottom-first, but that is the AGU's job — it counts <code>w_addr</code> down 3,2,1,0 during WLOAD. Do not pre-reverse them in the host. If you do, rows 0–2 come out wrong while row 3 still looks right (the all-ones activation row produces column sums, which are insensitive to row permutation) — which is exactly why this bug hides.
Every address you intend to read must be written	The memories have no reset. Unwritten addresses return X in simulation and undefined data on hardware.

10.3 Launching the job

```
set vio_num_vectors = N      ;# how many activation vectors (4 for the T5 demo)
set vio_act_base     = 0      ;# where in a_mem to start reading
set vio_out_base     = 0      ;# where in output_mem to start writing
pulse vio_start_lvl 1 then 0 ;# oneshot → one clock of `start`
```

The job then runs to completion with **no handshaking of any kind**: `num_vectors + 13` clocks — 17 clocks = 170 ns for `N = 4`. There is no ready/valid, no back-pressure, and nothing will stall to wait for you. By the time the *next* JTAG transaction arrives, the job finished long ago. That is precisely why `done_sticky` exists and why you poll it rather than `done`.

NUM_VECTORS = 0 IS SAFE

The FSM guards it: both WLOAD and IDLE exit to `(vec_n == 0) ? PH_DRAIN : PH_COMPUTE`. Without the guard, COMPUTE would compare against `8'hFF` and sit for 256 cycles issuing garbage writes. It is worth knowing this is handled, because a VIO probe left at its default is an easy way to launch a zero-vector job by accident.

11. Data packing — the lane and address rules

Every word is 32 bits carrying four signed int8 lanes, **lane 0 in the LOW byte**.

Memory	Depth × width	Word content	Address meaning
weight_mem	4 × 32 (LUTRAM)	Word r , lane $c = \mathbf{W}[r][c]$	Weight matrix row index, 0–3 (2-bit bus).
a_mem	256 × 32 (BRAM)	Word i , lane $r = \mathbf{A}[i][r]$	Activation vector index, 0–255 (8-bit bus).
output_mem	N × 128 (BRAM)	Word i , lane $c = \mathbf{C}[i][c]$ as int32	Result row index.

11.1 Worked packing — the T5 golden case

A (4×4)

[1 0 0 0]

[0 1 0 0]

[0 0 1 0]

[1 1 1 1]

×

W (4×4)

[1 2 3 4]

[5 6 7 8]

[9 10 11 12]

[13 14 15 16]

=

C = A × W (expected)

[1 2 3 4] ← A row0 selects W row0

[5 6 7 8] ← A row1 selects W row1

[9 10 11 12] ← A row2 selects W row2

[28 32 36 40] ← A row3 = column sums

weight_mem

addr 0 ← 0x04030201 (W row0 = 1 2 3 4)

addr 1 ← 0x08070605 (W row1 = 5 6 7 8)

addr 2 ← 0x0C0B0A09 (W row2 = 9..12)

addr 3 ← 0x100F0E0D (W row3 = 13..16)

▲

lane0 = LOW byte = 0x01 = W[0][0]

a_mem

addr 0 ← 0x00000001 (A row0 = 1 0 0 0)

addr 1 ← 0x00000100 (A row1 = 0 1 0 0)

addr 2 ← 0x00010000 (A row2 = 0 0 1 0)

addr 3 ← 0x01010101 (A row3 = 1 1 1 1)

▲

lane0 = LOW byte = A[i][0]

WHY T5 IS THE RIGHT VECTOR TO REPLAY ON HARDWARE

It is chosen so that

EVERY FAILURE MODE ANNOUNCES ITSELF DISTINCTLY

. The identity rows mean rows 0–2 of the result should literally reproduce the weight rows — so a transposed or lane-swapped read is obvious at a glance. The all-ones row 3 produces column sums 28 32 36 40, which are hand-checkable and are the one row insensitive to weight-row ordering. And it is the

SAME

vector that section T5 of `tb_accelerator_system_top` checks in simulation, so a hardware match is a direct simulation-to-silicon equivalence result, not a new claim.

12. Strategy for watching results

There are three independent observation channels, each answering a different question. Using all three is the methodology; using only the first is guesswork.

Channel	Question it answers	How it works
VIO probe_in — <code>o_rdata</code>	"Is the answer correct?"	Set <code>vio_o_addr</code> to a result row; read the 128-bit <code>o_rdata</code> ; unpack four int32 lanes. There is 1 clock of SRAM read latency between address and data — irrelevant here, because a JTAG round trip is about six orders of magnitude longer, so no wait is needed.
VIO probe_in — <code>status</code>	"Did it run at all, and is it ready?"	<code>busy</code> , <code>done_sticky</code> , <code>done_count</code> . <code>done_count</code> incrementing by exactly 1 per start is the health check; <code>busy</code> is the interlock.
ILA waveform	"Did it run <i>correctly</i> , cycle by cycle?"	Arm on <code>start_p == 1</code> , cause a run, and read the captured 17 cycles against the expected cycle map in §14. This is the only channel that can see <code>done</code> (a 10 ns pulse), the phase transitions, and the diagonal wavefront emerging.

12.1 Unpacking a result word correctly

The 128-bit word is MSB-first, so hex character positions map to lanes in **reverse**:

```
o_rdata[127:96] = chars 0..7   = column 3
o_rdata[ 95:64] = chars 8..15  = column 2
o_rdata[ 63:32] = chars 16..23 = column 1
o_rdata[ 31: 0] = chars 24..31 = column 0    ← lane 0 lives at the END of the string
```

Two traps:

- Vivado DROPS LEADING ZEROS, so a small result comes back as a short string. You must LEFT-PAD to 32 hex characters before slicing, or every lane shifts.
- Each lane is a SIGNED int32. Values $\geq 0x80000000$ must be sign-extended (subtract `0x1_0000_0000`), or negative results read as ~4 billion.

12.2 The Tcl semantics that catch everyone

Call	What it really does
<code>set_property</code> <code>OUTPUT_VALUE ...</code>	Stages the value locally in Vivado's memory only . Nothing has reached the FPGA yet.
<code>commit_hw_vio</code> <code><probe></code>	Pushes the staged values over JTAG . Forgetting this is the number-one VIO scripting bug — your script "works" and the hardware never sees a thing.
<code>refresh_hw_vio</code> <code><vio></code>	Pulls current values from the device . Reading <code>INPUT_VALUE</code> without a preceding refresh returns a stale cached snapshot from whenever the last refresh happened.
<code>get_property</code> <code>INPUT_VALUE</code> ...	Reads the cached snapshot. Only as fresh as your last refresh.
<code>OUTPUT_VALUE_RADIX</code> / <code>INPUT_VALUE_RADIX</code>	Set the radix before the value, or a hex string will be interpreted as binary and silently truncated.

13. Scripted operation — `vio_demo.tcl` dissected

Manual dashboard clicking is fine once. For a repeatable result the run is scripted. The script is structured defensively because Hardware Manager state is easy to get wrong:

Procedure	What it does and the failure it prevents
<code>accel_vio</code>	Connects and caches the VIO object. Explicitly selects the xc7z020* device rather than relying on one being current — a bare <code>get_hw_vios</code> silently returns nothing if you clicked elsewhere in the GUI, or if <code>arm_dap_0</code> is the current device. On failure it prints a four-point checklist rather than an opaque error.
<code>accel_probe {name dir}</code>	Resolves a probe by name. Vivado names probes after the attached net, but the exact string varies by version — <code>vio_w_wdata</code> in some, a hierarchical <code>vio_debug_top_i/vio_w_wdata</code> in others. So: exact tail match first, substring match as fallback, always filtered by direction so a value call can never land on the wrong probe, and on ambiguity it lists the candidates rather than silently picking one .
<code>accel_probes</code>	Prints every probe with direction and width — the first thing to run when a name lookup fails.
<code>accel_set / accel_get</code>	Set-radix → set-value → commit , and refresh → read. The two halves of §12.2, encapsulated so they cannot be forgotten.
<code>accel_write_w <addr> <hex></code>	One weight word: wdata → addr → we 1 → we 0.
<code>accel_write_a <addr> <hex></code>	One activation word, same pattern on the 8-bit address bus.
<code>accel_run <nvec></code>	Sets <code>num_vectors</code> / <code>act_base</code> / <code>out_base</code> , records done_count before , pulses start, then polls <code>done_sticky</code> up to 20 times. Times out with a diagnostic naming the two real causes (POR not released, <code>num_vectors</code> = 0) instead of hanging.
<code>accel_read <addr></code>	Sets <code>o_addr</code> , reads <code>o_rdata</code> , strips separators, left-pads to 32 hex chars , slices from the end backwards, and sign-extends each int32 . Returns col0-first.
<code>accel_demo</code>	The whole regression: load 4 weight words, load 4 activation words, run, read all 4 result rows, compare against the golden matrix , and print <code>PASS</code> or a per-row <code>MISMATCH exp=...</code> .

13.1 The output you are looking for

```

=== 4x4 systolic accelerator :: VIO demo (T5 golden case) ===
loading weights...
loading activations...
running (num_vectors = 4, 17 clocks = 170 ns)...
job done  (done_count 1 , was 0)

C = A x W :
[   1   2   3   4 ] OK
[   5   6   7   8 ] OK
[   9  10  11  12 ] OK
[  28  32  36  40 ] OK

PASS -- hardware matches the simulation golden model.

```

14. The expected ILA cycle map

This is the reference the captured waveform is checked against. Cycle 0 is the clock on which `start_p` is high; every number is a hard constant derived from the RTL, not an approximation — which is exactly what makes the capture **checkable**.

Cycle	phase	Phase name	Length	What you should see on the probes
0	0 → 1	IDLE → WLOAD	1	<code>start_p</code> high for exactly one clock; <code>busy</code> rises.
1–4	1	WLOAD	ROWS = 4	Weight rows driven into the array's north edge bottom-first : W[3], W[2], W[1], W[0]. If <code>agu_w_addr</code> is marked, it counts down 3,2,1,0 .
5–8	2	COMPUTE	N = 4	One dense activation vector per clock, <code>valid = 1</code> . A0@5, A1@6, A2@7, A3@8. This is the throughput claim — once the pipe is full it is one vector per clock . COMPUTE follows WLOAD directly : there is no FLUSH phase.
9–16	3	DRAIN	STORE_LATENCY = 8	The last vector is still in flight. Results land here: watch <code>result_flat</code> populate and, if marked, four <code>agu_out_we</code> strobes at cycles 13–16. The weight residue left in <code>psum_q</code> by WLOAD leaves the de-skewed result bus on cycle 12, one clock before the first strobe.
17	4	DONE	1	<code>done</code> pulses for exactly one clock. <code>busy</code> is already low. <code>done_count</code> increments by 1.

Total job length = ROWS + N + STORE_LATENCY + 1 = 4 + N + 8 + 1 = **N + 13**
Reusing the loaded weights (WLOAD skipped) = N + STORE_LATENCY + 1 = **N + 9**
For N = 4: **17 clocks = 170 ns**, or **13 clocks = 130 ns** reusing weights, at 100 MHz

TWO CHANGES FROM EARLIER REVISIONS — BOTH VISIBLE ON THE ILA

1. There is no FLUSH phase. The FSM used to insert `ROWS + 2 = 6` cycles between WLOAD and COMPUTE to push weight residue out of the `psum_q` registers. It was margin, not function — the residue clears the de-skewed result bus a full cycle before the first `out_we` fires — so it was removed, cutting 6 clocks from every job. Phase encodings shifted with it:

0 IDLE, 1 WLOAD, 2 COMPUTE, 3 DRAIN, 4 DONE

.

2. WLOAD is skipped when the weights have not changed. `accelerator_system_top` keeps an internal `weights_dirty` flag — set by reset and by any host write to `weight_mem` that reaches the SRAM, cleared on the last WLOAD cycle — and drives `controller.load_weights` with it. Run the same weights twice and the second job takes **13 CLOCKS INSTEAD OF 17**

.

THIS IS THE EASIEST THING TO CONFIRM ON THE ILA:

arm on `start_p`, run a job, then run it again without touching the weight probes — on the second capture the `phase` probe steps

0 → 2 DIRECTLY, WITH NO 1 IN BETWEEN

. Nothing in the host protocol changes and the module's port list is unchanged, so `fpga_top`, `vio_debug_top` and `accel_axi` all benefit untouched.

14.1 The latency chain, and where to see each term

#	Step	Adds	Running total	Visible as
1	Down a PE column (one register per row)	$ROWS - 1 = 3$	3	Bottom-row psum appears 3 cycles after the vector enters.
2	East to column c	$+c$	$3+c$	Column c 's value is ready at $n+3+c$ — the diagonal wavefront.
3	De-skew pipe ($COLS - 1 - c$)	$+(3-c)$	6 — independent of c	All four lanes of <code>result_flat</code> align on the same clock.
4	<code>a_mem</code> read latency	$+1$	7 = <code>RESULT_LATENCY</code>	Result valid relative to the address issued.
5	Write strobe, one cycle after data is observable	$+1$	8 = <code>STORE_LATENCY</code>	<code>agu_out_we</code> pulses, and DRAIN is exactly this long.

STEP 3 IS THE ELEGANT PART, AND IT IS WORTH POINTING AT IN A REVIEW

Column c 's result is ready at $3+c$ — a diagonal wavefront, four different arrival times. The de-skew buffer delays column c by $3-c$. The two cancel exactly, so the sum is

6 FOR EVERY COLUMN

and the whole result row emerges aligned on one clock. Seeing that alignment on the ILA is direct confirmation that the output skew buffer is doing its job.

15. Trigger recipes — what to look at and when

Goal	Trigger setup	What to read from the capture
Capture a whole job (the default)	<code>probe0 (start_p) == 1</code>	Everything in §14. Set the trigger position early in the window so you get pre-trigger context.
Confirm weight writes actually happened	<code>probe4 (w_we_p) == 1</code>	You should see exactly four 1-clock spikes , one per <code>accel_write_w</code> . More means the one-shot is broken; fewer means a write never committed.
Confirm activation writes	<code>probe5 (a_we_p) == 1</code>	Four spikes, with <code>probe6 (vio_a_addr)</code> stepping 0,1,2,3.
Catch the completion instant	<code>probe2 (done) == 1</code>	The 10 ns pulse the VIO can never see. Confirm <code>busy</code> is already low on that clock.
Measure phase lengths directly	<code>probe3 (phase)</code> transitions	Count clocks in each phase and compare to $4 / N / 8 / 1$. This is the cheapest possible validation of the controller on real silicon.
Watch the wavefront emerge	<code>probe0 == 1</code> , then scroll to DRAIN	<code>probe8 (result_flat)</code> — the four int32 lanes filling in and aligning.
Catch an intermittent failure across many runs	<code>probe9 (done_count)</code> at a specific value	Lets you trigger on the n -th job rather than the first — the way to find a failure that only appears after a sweep.
Prove the bottom-first weight rule	<code>probe3 (phase) == 1</code> with <code>agu_w_addr</code> marked	<code>agu_w_addr</code> counting 3,2,1,0 while the host wrote 0,1,2,3.

16. Parameters and reports checked at each stage

This is the audit trail. Each stage has specific artefacts, specific numbers, and a specific pass criterion.

16.1 Stage A — simulation (before any hardware)

What we check	Where	Pass criterion / achieved
Functional regression	QuestaSim, <code>verification_recheck/</code>	11/11 testbenches, 3 680 checks, 0 mismatches ✓
Golden vector T5	<code>tb_accelerator_system_top</code>	C = [1 2 3 4 / 5 6 7 8 / 9 10 11 12 / 28 32 36 40] ✓
Phase durations	<code>tb_controller</code>	WLOAD 4, COMPUTE N, DRAIN 8, DONE 1; total N+13 ✓
Wrapper elaboration	QuestaSim <code>vopt</code>	<code>fpga_top.sv</code> and <code>accel_axi.sv</code> elaborate clean ✓

16.2 Stage B — synthesis

Report / parameter	Command or GUI path	Pass criterion / measured
Messages — errors, critical warnings	<i>Messages</i> pane	0 errors, 0 warnings, 0 critical warnings for the accelerator OOC build ✓
Utilization	<code>report_utilization</code>	LUT 1 564 / 53 200 = 2.94 % ; FF 792 / 106 400 = 0.74 % ; BRAM 2.5 / 140 = 1.79 % ; DSP 0 / 220 ; LUTRAM 40 ✓
DSP inference	Same report	0 DSPs — the sixteen 8×8 multipliers went to LUT fabric. Expected, and the first lever if the clock is ever raised.
Memory inference	Same report	<code>a_mem / output_mem</code> → BRAM; <code>weight_mem</code> 4×32 → distributed LUTRAM (a deeper weight memory would be dead silicon) ✓
Harness survival	Netlist search for <code>u_os_start</code>	The three <code>oneshot</code> instances must exist and <code>start_p</code> must be driven by a flop , not directly by the VIO.
Clock definition	<code>report_clocks</code>	Exactly 1 clock, <code>sys_clk</code> , period 10.000 ns ✓
Constraint completeness	<code>check_timing</code>	Understand every category before proceeding.

16.3 Stage C — implementation (the sign-off stage — see [timing.pdf](#))

Report / parameter	Command	Pass criterion / measured
WNS — Worst Negative Slack	<code>report_timing_summary</code> on the implemented design	Must be > 0. Measured +0.714 ns (debug build) / +1.279 ns (accelerator alone) ✓
WHS — Worst Hold Slack		Must be > 0. Measured +0.045 ns / +0.263 ns ✓
WPWS — Worst Pulse Width Slack		Must be > 0. Measured +3.750 ns in both ✓
TNS / THS / TPWS	Same	All must be 0.000 ns. Measured 0.000 / 0.000 / 0.000 ✓
Failing endpoints	Same	Must be 0. Measured 0 of 5 181 (setup and hold), 0 of 2 950 (pulse width) ✓
The verdict sentence	Same	“All user specified timing constraints are met.” ✓
Worst-path detail	<code>report_timing - delay_type max - nworst 10</code>	Confirm the critical path is the expected one — the PE multiply-accumulate — and read the logic/route split.
Unconstrained paths	<code>report_timing_summary / check_timing</code>	Understand every entry. An unconstrained path is not analysed and is invisible in WNS.
DRC	<code>report_drc</code>	No blocking violations. The PL-only unconstrained-PS DRC is allowed explicitly in the XDC.
Methodology	<code>report_methodology</code>	Review all TIMING-* rules.
Post-implementation utilization	<code>report_utilization</code>	Includes the debug cores; confirms the ILA BRAM cost you chose.
Endpoint count as a sanity check	Timing summary	1 305 (no debug) → 5 181 (with VIO+ILA). The increase confirms the debug logic really is in the netlist.

16.4 Stage D — on hardware

What we check	Channel	Pass criterion
Device enumerates	Hardware Manager	<code>xc7z020_1</code> and <code>arm_dap_0</code> both visible under Auto Connect.
Debug cores present	Hardware pane	<code>hw_vio_1</code> and <code>hw_ila_1</code> under the device; no "debug hub not detected".
Probe inventory	<code>accel_probes</code>	17 probes with the widths of §6 — 6 IN, 11 OUT.
Reset released	<code>busy</code>	Reads 0 at rest. If it reads 1 forever, see §18.
Job launched	<code>done_count</code>	Increments by exactly 1 per start click.
Job completed	<code>done_sticky</code>	Reads 1 after the run, and is auto-cleared by the next start.
Numerical result	<code>o_rdata</code> rows 0–3	Exact match to the golden T5 matrix. <code>accel_demo</code> prints PASS.
Execution shape	ILA capture	Phase lengths 4 / 4 / 8 / 1; total 17 clocks; four <code>w_we_p</code> and four <code>a_we_p</code> spikes; <code>done</code> one clock wide with <code>busy</code> already low.
Bottom-first rule	ILA + <code>MARK_DEBUG</code>	<code>agu_w_addr</code> counts down 3,2,1,0 while the host wrote 0,1,2,3.
Store latency	ILA + <code>MARK_DEBUG</code>	<code>agu_out_we</code> strobes exactly 8 cycles after each activation address was issued.

17. The pass/fail criteria we actually apply

Compressed to the statements that must all be true before the implementation is called done:

FUNCTION	simulation: 11/11 TBs, 3 680 checks, 0 mismatches
FUNCTION	hardware: all 16 result values match the golden T5 matrix exactly
TIMING	WNS > 0, WHS > 0, WPWS > 0, TNS = THS = TPWS = 0.000
TIMING	0 failing endpoints, and the analysed endpoint count is plausible
TIMING	"All user specified timing constraints are met."
BEHAVIOUR	ILA phase lengths = 4 / N / 8 / 1, total N+13 clocks
BEHAVIOUR	done_count increments by exactly 1 per start; done is 1 clock wide
INTEGRITY	the 13 RTL files are byte-identical to the simulated versions
RESOURCE	utilisation within budget: 2.94 % LUT, 0.74 % FF, 1.79 % BRAM, 0 DSP
BUILD	0 synthesis errors, 0 critical warnings; DRC and methodology reviewed

18. Failure modes and what each report says

Symptom	Most likely cause	Fix / which report to open
"The debug hub core was not detected"	The hub does not know its clock frequency, or JTAG is too fast relative to the design clock	Confirm the <code>dbg_hub</code> block in <code>vio_debug.xdc</code> ran. Set <code>C_CLK_INPUT_FREQ_HZ 100000000</code> and <code>C_ENABLE_CLK_DIVIDER false</code> . Also try lowering the JTAG frequency: right-click target → <i>Open Target Properties</i> → 6 MHz or less.
No <code>hw_vio_1</code> / <code>hw_ila_1</code> under the device	The <code>.ltx</code> probes file was not loaded with the bitstream	Re-run <i>Program Device</i> and set <i>Debug probes file</i> to <code><project>.runs/impl_1/vio_debug_top.ltx</code> .
Device does not appear at all	Cable drivers, wrong USB port, or boot-mode jumpers	Use J17 , not J14. <code>PG00D</code> lit. JP7–JP11 all to GND. Reinstall Vivado cable drivers.
<code>busy</code> reads 1 forever	Reset never released, or <code>start</code> stuck high	The POR is 65 536 clocks — it releases in 655 μ s, so this is never a "wait longer" problem. Check <code>vio_start_lv1</code> is 0 on the dashboard, and confirm the <code>oneshot</code> instances survived synthesis (search the netlist for <code>u_os_start</code>).
<code>done_count</code> jumps by thousands per click	The one-shot was optimised away or bypassed	Confirm <code>oneshot</code> is present in the netlist and that the elaborated design shows <code>start_p</code> driven by a flop , not directly by the VIO. This is mismatch 1 of \$4, live.
<code>done_sticky</code> never sets	The job never launched — <code>start</code> arrived while <code>busy</code> was high, so it was ignored	Read <code>busy</code> first; it must be 0. Confirm <code>done_count</code> did not move either.
All results read 0	Host writes masked because <code>busy</code> was 1, or the writes never pulsed	Re-load with <code>busy == 0</code> . Add an ILA trigger on <code>w_we_p == 1</code> and confirm you see exactly four 1-clock spikes.
Result is transposed or lane-scrambled	Wrong lane order in the packed word	Lane 0 is the LOW byte. <code>weight_mem[r] lane c = W[r][c]</code> ; <code>a_mem[i] lane r = A[i][r]</code> . Re-check against §11.
Row 3 reads 28 32 36 40 but rows 0–2 are wrong	Weight rows written in reverse order by the host	Write <code>weight_mem[0..3]</code> in natural order — the bottom-first shift is the AGU's job. Row 3 is the column sum and is insensitive to row permutation, which is exactly why this failure hides there. A textbook example of why the golden vector was chosen as it was.
A small result reads with shifted lanes	Vivado dropped leading zeros in the hex string	Left-pad to 32 hex characters before slicing (§12.1). <code>accel_read</code> does this.
Negative results read as ≈ 4 billion	Missing int32 sign extension	Subtract <code>0x1_0000_0000</code> from any lane $\geq$ <code>0x80000000</code> .
A Tcl script "runs" but nothing changes on the board	Missing <code>commit_hw_vio</code>	<code>set_property</code> only stages locally. See §12.2.
Readback values never change	Missing <code>refresh_hw_vio</code>	You are reading a stale cached snapshot. See §12.2.
Ambiguous probe name error from Tcl	Vivado's probe naming varies by version	Run <code>accel_probes</code> and use a longer, unambiguous name.
Results read as X in simulation but fine on hardware	The memories have no reset	Expected behaviour. Write every address you intend to read.
Timing fails after adding debug cores	ILA depth and wide probes cost routing	Reduce depth 1024 → 256; drop probes you are not reading; drop <code>-flatten_hierarchy none</code> . Last resort: MMCM to 50 MHz — nothing here depends on throughput — and update <code>C_CLK_INPUT_FREQ_HZ</code> to match, or you trade a timing failure for a "debug hub not detected".

19. Alternative methods 1, 3 and 4 — when to use them

19.1 Method 1 — switches and LEDs (`fpga_top.sv`)

A console FSM around the accelerator: enter each int8 byte on `SW[7:0]`, commit with **BTNC** (the pointer auto-advances), 16 commits for weights then 16 for activations; **BTNU** runs the job; then `SW[5:4]` selects the result row and `SW[3:0]` selects the byte within that row's 128-bit word, and the LEDs display it. **BTND** resets entry. Use it when you want a self-contained board demo with no PC attached at run time. Sanity check: with T5 loaded, `SW[5:4] = 11`, `SW[3:0] = 0000` → LEDs `00011100` = 28.

19.2 Method 3 — USB-UART text console (`accel_axi.sv` + `accel_monitor.c`)

On the ZedBoard the USB-UART is a **PS** resource (UART1, MIO48/49), so this needs a minimal Zynq PS with `accel_axi` bridging AXI-Lite to the accelerator. Data path: terminal → USB → PS UART1 → ARM → AXI-Lite → `accel_axi` → accelerator (PL). Register map (byte address = index × 4): 0 NUMVEC · 1 ACT_BASE · 2 OUT_BASE · 3 W_WDATA · 4 W_COMMIT · 5 A_WDATA · 6 A_COMMIT · 7 START · 8 O_ADDR · 9 STATUS(busy,done,phase) · 10–13 ORD0–3. Note the same architectural pattern as the VIO harness: **a commit register that turns a bus write into a one-cycle strobe, and a sticky-done bit** — the two mismatches of \$4 recur, because they are inherent to bridging a slow host to a fast pipeline. Use it when a human needs to type matrices and read results in text. Terminal: 115200 8-N-1, no flow control.

19.3 Method 4 — full PS/AXI integration

What you add on top of Method 3 for real workloads: **AXI-DMA** to stream a block of activation vectors into `a_mem` and read the result block back instead of one word per register write; **a done-interrupt** routed to `IRQ_F2P` and enabled in the PS7 GIC so software is event-driven instead of polling the sticky bit; **Linux/UIO** under PetaLinux with an `mmap` ed register block; and **benchmarking** with up to 256 activation vectors (the depth of `a_mem`) — once the pipe is full it is **one vector per clock**.

20. Sign-off checklist

☐	Item
☐	The 13 RTL files are unmodified — byte-identical to the simulated versions.
☐	Simulation baseline current: 11/11 testbenches, 3 680 checks, 0 mismatches.
☐	<code>vio_0</code> generated with 6 probe_in / 11 probe_out at the widths in §6; <code>vio_num_vectors</code> initial value <code>0x04</code> .
☐	<code>ila_0</code> generated with 10 probes at the widths in §7; depth chosen deliberately (256 recommended).
☐	<code>vio_debug_top</code> is the top module; no black-box IP.
☐	Clock pin verified against the Digilent master XDC for the actual board revision .
☐	<code>create_clock -period 10.000</code> present; <code>report_clocks</code> shows exactly 1 clock.
☐	<code>check_timing</code> run and every category understood.
☐	Synthesis: 0 errors, 0 critical warnings.
☐	The three <code>oneshot</code> instances exist in the netlist; <code>start_p</code> is flop-driven.
☐	Utilization reviewed: LUT 2.94 %, FF 0.74 %, BRAM 1.79 %, DSP 0.
☐	Post-route timing summary: WNS, WHS, WPWS all positive; TNS = THS = TPWS = 0.000; 0 failing endpoints .
☐	“All user specified timing constraints are met.” present in the report.
☐	Worst path inspected and is the expected one (PE multiply-accumulate).
☐	DRC and methodology reports reviewed.
☐	<code>dbg_hub</code> properties set: <code>C_CLK_INPUT_FREQ_HZ 100000000</code> , <code>C_ENABLE_CLK_DIVIDER false</code> .
☐	Bitstream generated; <code>.ltx</code> path noted.
☐	Board jumpers JP7–JP11 to GND; cable on J17; PG00D lit.
☐	Programmed <code>xc7z020_1</code> (not <code>arm_dap_0</code>) with the <code>.ltx</code> loaded.
☐	<code>hw_vio_1</code> and <code>hw_ila_1</code> both visible; <code>accel_probes</code> lists 17 probes.
☐	<code>busy</code> reads 0 at rest (POR released).
☐	<code>accel_demo</code> prints PASS — all 16 values match the golden T5 matrix.
☐	<code>done_count</code> increments by exactly 1 per start.
☐	ILA capture reviewed against the §14 cycle map: phases 4 / N / 8 / 1, total N+13 clocks.
☐	Four <code>w_we_p</code> and four <code>a_we_p</code> spikes, each exactly 1 clock wide.
☐	<code>done</code> observed as a 1-clock pulse with <code>busy</code> already low.

Sources: harness `FPGA_IMPL/rtl/vio_debug_top.sv`; constraints `FPGA_IMPL/vio_debug.xdc`; automation `FPGA_IMPL/vio_demo.tcl`; VIO IP configuration and Design Timing Summary from `synthesis_screenshots/`; resource and out-of-context timing figures from `VIO_ILA.pdf` §15.

Design reference: `context_mkdown/` — chunk 01 (processing element), chunk 06 (memories and lane layout), chunk 07 (controller FSM), chunk 08 (AGU), chunk 11 (L3 wrapper and host protocol), chunk 12 (dataflow and timing derivations), chunk 13 (worked example / T5 golden case).

Functional baseline: 11/11 testbenches, 3 680 checks, 0 mismatches (`verification_recheck/`).

Companion documents: `timing.pdf` (timing, WNS and closure in full), `VIO_ILA.pdf` (Method 2 reference), `USB_UART.pdf` (Method 3 reference), `ZedBoard_Implementation_Guide.html` (all four methods).